GUIDE D'EXPOSÉ ORAL & SCRIPT DU PRÉSENTATEUR

Les Systèmes d'Exploitation Multiprocesseurs : Fiches Mémos, Minutage et Script Mot-à-Mot

Support pour présentation orale face à un jury ou un auditoire

Introduction : L'impasse physique et l'avènement du multicœur **Durée : ~3 min**

FICHE MÉMO VISUELLE (POINTS CLÉS À PROJETER OU GARDER EN TÊTE)

- **Loi de Moore (1965) :** Le nombre de transistors double tous les 2 ans. Toujours valide aujourd'hui !
- **Échelle de Dennard (1974) :** Densité de puissance constante lors de la miniaturisation → hausse de fréquence gratuite.
- **La rupture de 2005 (Les 3 murs) :**
  - *Power Wall :* Fuites électriques, surchauffe ingérable.
  - *Frequency Wall :* Plafond à 3-4 GHz.
  - *Memory Wall :* Le processeur attend la RAM.
- **Conséquence majeure :** Fin du "Free Lunch". On bascule vers le multicœur. L'OS devient l'élément clé du parallélisme.

SCRIPT ORAL MOT-À-MOT (CE QUE VOUS DITES À VOIX HAUTE)

"Bonjour à tous. Aujourd'hui, nous allons aborder une transformation majeure qui a façonné toute l'informatique moderne : le passage aux systèmes d'exploitation multiprocesseurs.

Pendant près de trente ans, les développeurs ont bénéficié de ce que Herb Sutter a appelé le 'repas gratuit' ou *free lunch*. Un programme écrit en 1990 s'exécutait deux fois plus vite en 1992 sans retoucher à la moindre ligne de code. Pourquoi ? Parce que la loi de Moore doublait le nombre de transistors, et l'échelle de Dennard permettait d'augmenter la fréquence d'horloge sans que la puce ne brûle.

Mais vers 2005, ce modèle s'est effondré sur trois murs physiques : le mur de la puissance, le mur de la fréquence et le mur de la mémoire. À force de rapetisser les transistors, l'électricité s'est mise à fuiter, créant une chaleur impossible à dissiper. La fréquence s'est bloquée net autour de 3 à 4 Gigahertz.

Ne pouvant plus construire une seule voiture roulant à 1000 km/h, les ingénieurs ont décidé d'envoyer un convoi de dix camions roulant à 100 km/h. C'est la naissance du multicœur. Mais un camionneur a besoin d'un régulateur de trafic : c'est le rôle fondamental du système d'exploitation multiprocesseur, que nous allons explorer ensemble."

Partie 1 : Modèles d'OS et Organisation du Noyau **Durée : ~4 min**

FICHE MÉMO VISUELLE

- **AMP (Asymétrique) :** Un processeur maître (exécute le noyau), des processeurs esclaves (programmes utilisateur).
  - *Analogie :* Un restaurant avec un seul chef de cuisine pour toutes les commandes.
  - *Limite :* Goulot d'étranglement, point individuel de défaillance.
- **SMP (Symétrique) :** Tous les processeurs sont égaux et peuvent exécuter le noyau.
  - *Problème :* Risque de corruption de la mémoire du noyau.
- **Évolution de la protection :**
  - *Verrou Global (BKL) :* Simple mais bloque le système (un seul processeur en mode noyau).
  - *Verrous fins :* Un verrou par structure de données. Très efficace mais complexe (exemple du nettoyage du BKL dans Linux 2.6.39).

SCRIPT ORAL MOT-À-MOT

"Rentrons dans le vif du sujet : comment organise-t-on le noyau d'un OS sur une telle machine ?

Historiquement, la solution la plus simple a été le modèle asymétrique, ou AMP. Un processeur est désigné comme le maître et exécute tout le noyau. Les autres processeurs sont des esclaves qui font tourner les applications et supplient le maître à chaque besoin. C'est l'analogie d'une cuisine de restaurant où plusieurs commis découpent des légumes, mais où un seul chef détient les clés de la réserve. Le problème est évident : le maître sature rapidement, et s'il tombe en panne, toute la machine s'arrête.

C'est pour cela que tous les systèmes modernes utilise le modèle symétrique, ou SMP. Ici, tous les processeurs sont égaux et chacun peut exécuter le noyau sur place. C'est formidable pour les performances, mais cela crée un danger mortel : deux cœurs qui modifient en même temps la liste des processus en mémoire vont la corrompre.

Pour éviter cela, les pionniers ont d'abord inventé le 'Big Kernel Lock', un gros verrou unique. C'était comme mettre un seul tampon officiel dans une mairie de dix guichets : un seul employé travaille pendant que neuf attendent. Il a fallu des décennies d'ingénierie, notamment dans le noyau Linux entre 1996 et 2011, pour découper ce gros verrou en des milliers de verrous ultra-fins, protégeant chaque donnée individuellement."

Partie 2 : L'Ordonnancement des Processus **Durée : ~4 min**

FICHE MÉMO VISUELLE

- **File globale vs Files locales :**
  - *File globale :* Équilibrage naturel, mais grosse contention sur le verrou et perte de cache.
  - *Files locales par CPU :* Pas de conflit, excellente utilisation du cache, mais risque de déséquilibre.
- **Équilibrage de charge :**
  - *Migration poussée (Push) :* Un sous-système rééquilibre les tâches.
  - *Migration tirée (Pull / Work Stealing) :* Un cœur inactif vole du travail à son voisin.
- **Affinité & NUMA :**
  - *Affinité processeur :* Garder un processus sur son cœur pour garder le cache L1/L2 "chaud".
  - *Architecture NUMA :* Accéder à la RAM locale est rapide, la RAM distante est lente. L'ordonnanceur doit grouper calcul et mémoire sur le même nœud.

SCRIPT ORAL MOT-À-MOT

"Une fois le noyau sécurisé, comment répartit-on les programmes sur les différents cœurs ? C'est le rôle de l'ordonnanceur.

Au début, on utilisait une file d'attente globale unique. Imaginez la file unique d'un guichet de banque : dès qu'un guichet se libère, la personne en tête avance. C'est parfait pour l'équilibrage, mais lorsque vous avez 64 cœurs, tous les processeurs se battent violemment pour accéder au verrou de cette file unique.

Les OS modernes préfèrent donc des files d'attente locales sur chaque cœur, comme les caisses de supermarché. C'est très fluide, mais si un client devant vous a un caddie débordant, votre file bloque pendant que la caisse voisine est vide. Pour corriger cela, l'OS utilise le 'work stealing' ou vol de travail : un cœur inactif va scruter la file de son voisin et lui dérober quelques tâches.

Cependant, déplacer une tâche coûte cher car la mémoire cache devient 'froide'. L'OS cherche donc à maintenir l'affinité processeur. Sur des serveurs à plusieurs sockets, appelés architectures NUMA, ce principe devient vital : aller chercher des données dans la RAM rattachée à un processeur distant est aussi lent que d'envoyer un coursier dans une autre ville. L'ordonnanceur doit donc impérativement garder le processus et sa mémoire sur le même nœud physique."

Partie 3 : Synchronisation et Pièges Logiciels **Durée : ~4 min**

FICHE MÉMO VISUELLE

- **Condition de concurrence (Race Condition) :** Deux cœurs modifient la même variable en même temps.
  - Incapacité de la désactivation des interruptions en SMP.
  - Base matérielle : Instructions atomiques (Test-and-Set, Compare-and-Swap).
- **Outils de verrouillage :**
  - *Spinlock :* Attente active (boucle). Rapide mais brûle du CPU.
  - *Mutex :* Maintien en sommeil. Économe en CPU mais coûteux en changement de contexte.
- **Pièges classiques :**
  - *Deadlock (Interblocage) :* Blocage mutuel (analogie du carrefour à 4 sens de priorité).
  - *False Sharing (Faux partage) :* Deux variables indépendantes sur la même ligne de cache de 64 octets provoquant un effondrement des performances.

SCRIPT ORAL MOT-À-MOT

"Mais le plus grand danger du multiprocessage, c'est la concurrence. Si deux cœurs exécutent simultanément l'instruction 'compteur + 1', ils lient la même valeur initiale et une incrémentation est perdue.

Sur un monoprocesseur, il suffisait de couper les interruptions pour être tranquille. En SMP, c'est inutile : couper les interruptions sur un cœur n'empêche pas le cœur d'à côté de continuer de tourner ! Il a fallu créer des instructions matérielles atomiques, comme le Compare-and-Swap.

À partir de là, l'OS propose deux grands outils : le Spinlock et le Mutex. Le Spinlock, c'est comme faire le piquet devant une cabine d'essayage en frappant à la porte toutes les secondes. Si la personne sort dans deux secondes, c'est très rapide. Mais si elle y reste dix minutes, vous avez gaspillé votre énergie. Le Mutex, lui, consiste à aller s'asseoir dans la salle d'attente. C'est plus sage, mais le temps de se lever et d'aller s'asseoir coûte un changement de contexte.

Et attention aux pièges ! Le Deadlock est le plus connu : quatre voitures arrivent en même temps sur un carrefour à quatre sens de priorité, chacune cédant le passage à sa droite. Tout le monde reste figé à jamais. Un autre piège vicieux est le 'faux partage' : si deux variables totalement indépendantes se retrouvent par malchance côte à côte dans la même ligne de cache de 64 octets, les cœurs vont se disputer cette ligne en permanence, détruisant les performances sans aucune erreur de logique."

Partie 4 : Limites Théoriques et Évolution **Durée : ~3 min**

FICHE MÉMO VISUELLE

- **Contention de verrous :** Plus on a de cœurs, plus le temps passé à attendre les verrous augmente.
- **Loi d'Amdahl :** Speedup = 1 / [(1-p) + p/n].
  - La fraction séquentielle (1-p) fixe le plafond de verre infranchissable.
  - Même avec 95% de code parallèle, le gain maximal absolu avec une infinité de cœurs est de 20.
- **Ordonnancement asymétrique moderne (big.LITTLE) :**
  - Alliance de cœurs de Performance (P-cores) et d'Efficacité (E-cores).
  - L'OS (Energy Aware Scheduler) doit placer intelligemment les tâches selon leur charge et l'énergie disponible.

SCRIPT ORAL MOT-À-MOT

"Arrivé à ce stade, une question naïve se pose : pourquoi ne pas mettre 1000 cœurs sur nos puces ? C'est ici que la loi mathématique d'Amdahl nous rattrape.

Amdahl nous démontre que le gain de performance est limité par la partie séquentielle du programme, c'est-à-dire les sections critiques où l'on est obligé d'attendre un verrou. Faites le calcul : si votre programme est parallélisable à 95 %, il reste 5 % de code séquentiel. Même en achetant un ordinateur avec un nombre infini de processeurs, votre accélération maximale absolue ne pourra JAMAIS dépasser 20 fois la vitesse initiale !

C'est pour cela que les systèmes récents changent d'approche. Regardez vos smartphones ou les puces récentes d'Apple et d'Intel : elles adoptent l'architecture big.LITTLE. On mélange sur la même puce de très gros cœurs ultra-rapides et de petits cœurs très économes en énergie. L'ordonnanceur moderne ne traite plus les cœurs à égalité : il envoie la musique de fond sur un petit cœur économe, et réserve le gros cœur puissant au jeu vidéo que vous venez de lancer."

Conclusion & Questions/Réponses **Durée : ~2 min**

SCRIPT ORAL DE CONCLUSION

"Pour boucler notre tour d'horizon, l'arrivée des systèmes multiprocesseurs n'a pas été un choix de confort, mais une obligation physique dictée par l'arrêt de l'échelle de Dennard. Les systèmes d'exploitation ont dû se réinventer complètement : de l'asymétrie initiale vers le SMP, de la file unique vers les structures NUMA, et de la symétrie parfaite vers l'ordonnancement hybride hétérogène.

Aujourd'hui, une nouvelle frontière s'ouvre avec l'intégration massive des accélérateurs spécialisés comme les GPU et les puces d'IA. L'OS de demain sera avant tout le chef d'orchestre d'une mosaïque de puces spécialisées. Je vous remercie de votre attention et je suis prêt à répondre à vos questions."

Préparation aux questions pièges du jury / de l'auditoire

Q1 : Quelle est la différence exacte entre la loi de Moore et l'échelle de Dennard ?

Réponse concise : La loi de Moore est une observation sur la densité d'intégration (le nombre de transistors double tous les 2 ans). L'échelle de Dennard concernait la physique de l'électricité : la densité de puissance restait constante en miniaturisant, permettant de monter en fréquence. La loi de Moore continue de s'appliquer, tandis que l'échelle de Dennard s'est arrêtée vers 2005.

Q2 : Pourquoi ne pas remplacer tous les verrous par du Spinlock pour éviter les changements de contexte ?

Réponse concise : Le Spinlock effectue une attente active (100% de CPU consommé dans une boucle). Si la section critique retenue par un autre cœur prend du temps (ex: lecture sur disque), le cœur qui attend brûle de l'énergie et chauffe pour rien. Le Spinlock n'est rentable que pour des attentes de quelques nanosecondes dans le noyau.

Q3 : Qu'est-ce que le principe RCU (Read-Copy-Update) dans Linux ?

Réponse concise : RCU est un mécanisme de synchronisation sans verrou pour les structures lues très souvent et modifiées rarement. Les lecteurs accèdent aux données directement sans aucun verrou. Lors d'une écriture, l'OS crée une copie de la structure, la modifie, puis bascule le pointeur de manière atomique, avant de supprimer l'ancienne version une fois que tous les lecteurs ont terminé.